Electric Vehicle Charging Monitoring System

• Project Overview

The **Smart Electric Vehicle Charging Intelligence System** is a data analytics project developed to study the usage patterns of electric vehicle charging networks. It integrates data related to charging sessions, charging stations, vehicles, and cities to understand energy consumption and infrastructure performance.

The project uses **SQL** for organizing the database, **Python** for data processing and analysis, and **Power BI** for building interactive dashboards. By analyzing charging duration, energy usage, costs, and grid load, the system reveals important trends in EV charging behaviour. The insights generated can help improve charging efficiency and support better planning of EV infrastructure.

• Dataset Summary

The dataset used in this project is structured into four primary tables, each representing different components of the electric vehicle charging ecosystem. These tables contain information related to charging sessions, charging infrastructure, city characteristics, and electric vehicle specifications.

1. Proj1_Charging_Slots [3007 unique row]

Columns(11) : session_id, vehicle_id, station_id, city_id, start_time, session_duration_minutes, energy_consumed_kwh, charging_cost, grid_load_percentage, renewable_ratio, peak_hour_flag

2. Proj1_Charging_Stations [1009 unique rows]

Columns(7) : station_id, city_id, operator_name, latitude, longitude, station_capacity_kw, total_ports, fast_ports

3. Proj1_City

Columns(7) : city_id, city_name, population, avg_temperature, ev_adoption_rate, renewable_energy_percent, base_price_per_kwh

4. Proj1_Vehicles

Columns(10) : vehicle_id, vehicle_type, manufacturer, battery_capacity_kwh, efficiency_km_per_kwh, segment, capacity, company_image, efficiency_insight, top_session_duration_insight.

Key Features of the Dataset

- Stores detailed records of EV charging sessions, including energy consumption, charging duration, and associated costs.
- Provides information about charging station infrastructure such as station capacity and the number of available charging ports.
- Includes city-level data such as population, temperature, EV adoption rate, and renewable energy usage to support broader contextual analysis.
- Contains vehicle-related attributes including manufacturer details, battery capacity, and efficiency metrics for comparative performance analysis.

• Exploratory Data Analysis using Python

1. **Library Imports:** The code imports essential Python libraries such as **Pandas** and **NumPy** for data manipulation, **Matplotlib** and **Seaborn** for data visualization, and **Scikit-learn** modules for data scaling and anomaly detection.

Environment Configuration: Display settings are adjusted using pandas.set_option() to ensure all dataset columns are visible during analysis.

[1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest

pd.set_option('display.max_columns', None)
sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (10,6)
```

2. Loading the files :

[2]:

```
cities = pd.read_csv("Proj1_City.csv")
stations = pd.read_csv("Proj1_Charging_Stations.csv")
vehicles = pd.read_csv("Proj1_Vechiles.csv")
sessions = pd.read_csv("Proj1_Charging_Slots.csv")

print("Files Loaded Successfully")
```

```
Files Loaded Successfully
```

3. Checking null values :

[4]:
```
print("========== BEFORE CLEANING ==========")

print("\nNull Values(sessions):")
print(sessions.isnull().sum())

print("\nNull Values(stations):")
print(stations.isnull().sum())
```

```
========== BEFORE CLEANING ==========

Null Values(sessions):
session_id                  0
station_id                  0
city_id                     0
vehicle_id                  0
start_time                  0
session_duration_minutes    0
energy_consumed_kwh         6
charging_cost               4
peak_hour_flag              0
grid_load_percentage        0
renewable_ratio            13
dtype: int64

Null Values(stations):
station_id            0
city_id               0
latitude              0
longitude             0
total_ports           0
fast_ports            0
station_capacity_kw   0
operator_name         0
dtype: int64
```

4. Cleaning :

sessions.isnull().sum()

After cleaning :

[6]:
```
sessions['energy_consumed_kwh'] = sessions['energy_consumed_kwh'].fillna(
    sessions['energy_consumed_kwh'].median()
)

sessions['charging_cost'] = sessions['charging_cost'].fillna(
    sessions['charging_cost'].median()
)

sessions['renewable_ratio'] = sessions['renewable_ratio'].fillna(
    sessions['renewable_ratio'].median()
)
```

[7]:
```
nulls_after = sessions.isnull().sum()
print("\nNull Values AFTER:\n", nulls_after)
```

```
Null Values AFTER:
 session_id                  0
station_id                  0
city_id                     0
vehicle_id                  0
start_time                  0
session_duration_minutes    0
energy_consumed_kwh         0
charging_cost               0
peak_hour_flag              0
grid_load_percentage        0
renewable_ratio             0
dtype: int64
```

5. Duplicate cleaning & Processing :

   Sessions.duplicated().sum()

   Stations = stations.drop_duplicates()

[8]:

```
print("\nDuplicate Rows in sessions:")
print(sessions.duplicated().sum())

print("\nDuplicate Rows in Stations:")
print(stations.duplicated().sum())
```

```
Duplicate Rows in sessions:
6

Duplicate Rows in Stations:
8
```

[10]:

```
stations = stations.drop_duplicates()
sessions = sessions.drop_duplicates()

print("\nDuplicate Rows in sessions:")
print(sessions.duplicated().sum())

print("\nDuplicate Rows in Stations:")
print(stations.duplicated().sum())
```

```
Duplicate Rows in sessions:
0

Duplicate Rows in Stations:
0
```

6. **Text Cleaning:** The vehicle_type column is cleaned using str.strip() to remove extra spaces from the values.

   Standardization: This ensures that all vehicle type entries follow a consistent format.

   Verification: The cleaned data is checked again to confirm that the values are properly standardized.

[12]:

```
print("Before Cleaning:")
print(vehicles['vehicle_type'].value_counts())
vehicles['vehicle_type'] = vehicles['vehicle_type'].str.strip().str.lower()
print("\nAfter Cleaning:")
print(vehicles['vehicle_type'].value_counts())
```

```
Before Cleaning:
vehicle_type
cars          16
motorbikes     7
bus            4
BUS            2
MOTORBIKES     1
Name: count, dtype: int64

After Cleaning:
vehicle_type
cars          16
motorbikes     8
bus            6
Name: count, dtype: int64
```

7. **Outlier Detection:** The code calculates the first quartile (Q1) and third quartile (Q3) of the energy_consumed_kwh column to determine the Interquartile Range (IQR).

   Outlier Filtering: Using the IQR method, lower and upper boundaries are defined to identify abnormal energy consumption values in the dataset.

[17]:

```
Q1 = sessions['energy_consumed_kwh'].quantile(0.25)
Q3 = sessions['energy_consumed_kwh'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

before_count = len(sessions)

sessions = sessions[
    (sessions['energy_consumed_kwh'] >= lower) &
    (sessions['energy_consumed_kwh'] <= upper)
]

after_count = len(sessions)

print("Outliers Removed:", before_count - after_count)
```

```
Outliers Removed: 9
```

8. **Data Check:** value_counts() is used to examine the distribution of values in the vehicle_type column before cleaning.

   Data Cleaning: Text values are standardized using functions like str.lower() to correct inconsistent formats.

   Verification: value_counts() is applied again to confirm that the categories are cleaned and consistent.

[11]:

```
print("Vehicle Types BEFORE Cleaning:")
print(vehicles['vehicle_type'].value_counts())
```

```
Vehicle Types BEFORE Cleaning:
vehicle_type
cars          16
motorbikes     7
bus            4
BUS            2
MOTORBIKES     1
Name: count, dtype: int64
```

[12]:

```
vehicles['vehicle_type'] = (
    vehicles['vehicle_type']
    .str.lower()
    .str.strip()
)

print("\nVehicle Types AFTER Cleaning:")
print(vehicles['vehicle_type'].value_counts())
```

```
Vehicle Types AFTER Cleaning:
vehicle_type
cars          16
motorbikes     8
bus            6
Name: count, dtype: int64
```

Maps and Plots

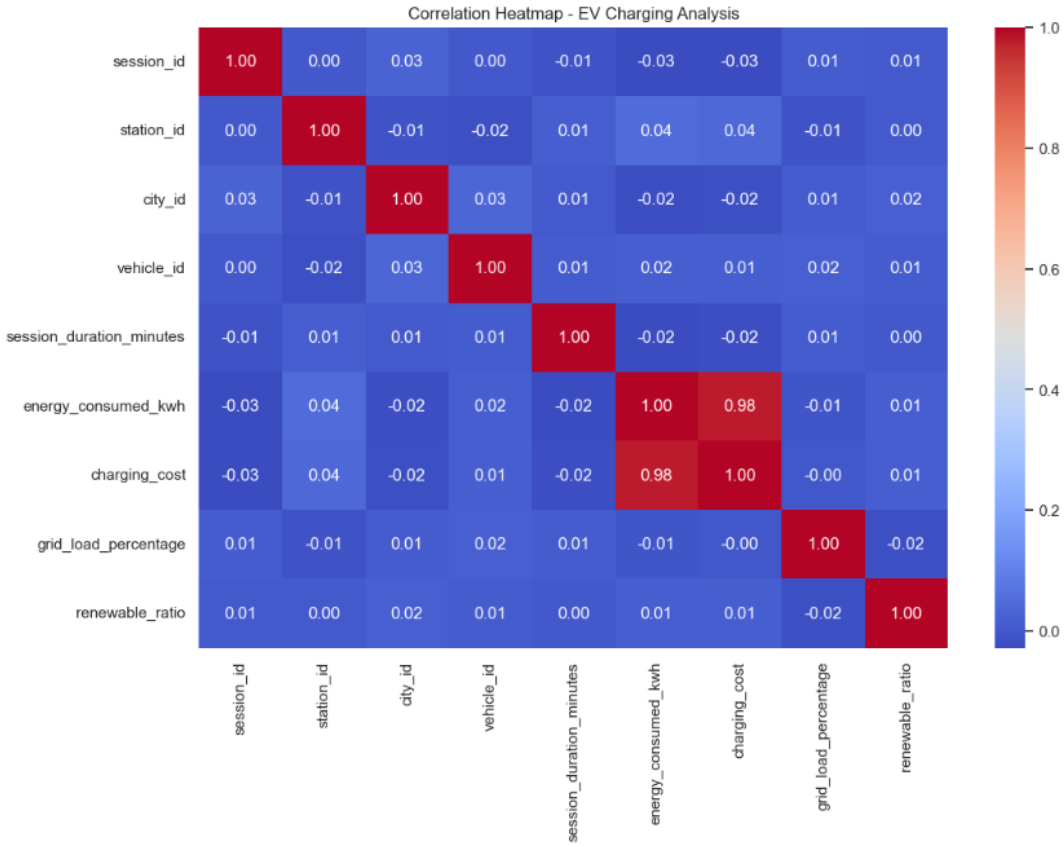
9. **Numeric Selection:** Numeric columns are selected using select_dtypes(include='number').

   Correlation Calculation: Relationships between variables are computed using corr().

   Visualization: sns.heatmap() is used to display the correlation matrix visually.

[19]:

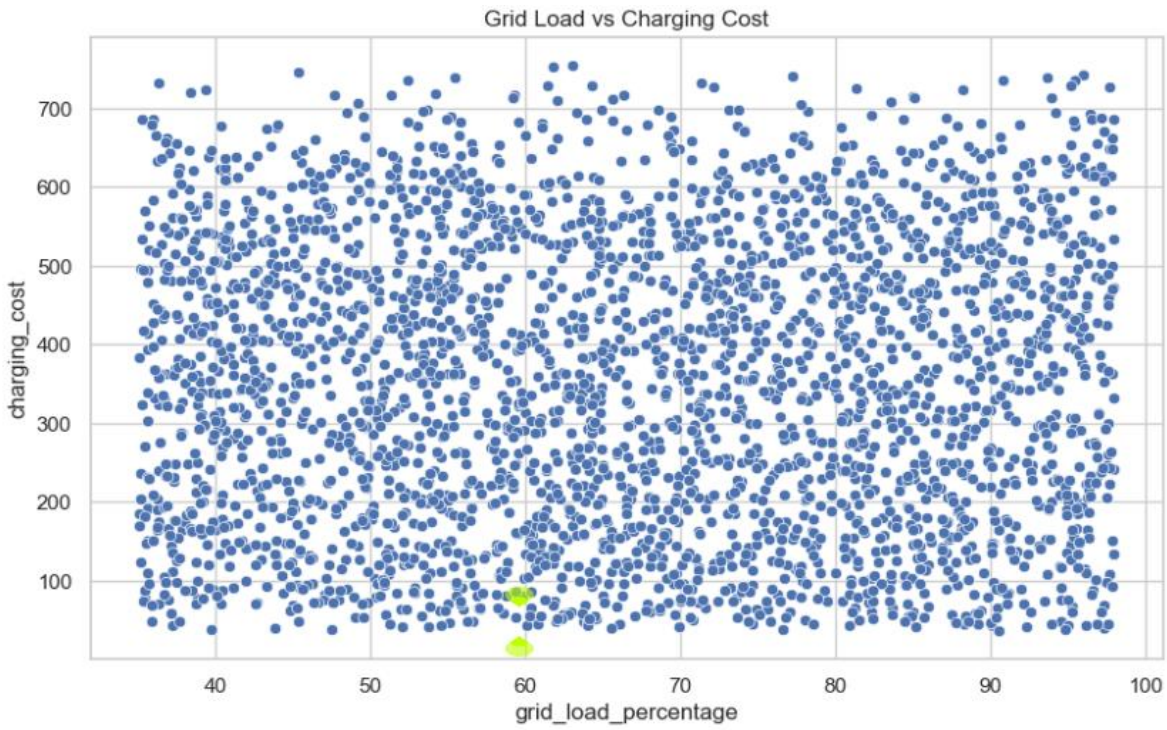
```
# HEATMAP
numeric_cols = sessions.select_dtypes(include='number')

plt.figure(figsize=(12,8))
sns.heatmap(numeric_cols.corr(),
            annot=True,
            cmap='coolwarm',
            fmt=".2f")

plt.title("Correlation Heatmap - EV Charging Analysis")
plt.show()
```



10. Grid Load Percentage

[20]:

```
# Revenue vs Grid Load
sns.scatterplot(x='grid_load_percentage',
                y='charging_cost',
                data=sessions)

plt.title("Grid Load vs Charging Cost")
plt.show()
```



More Imports

11. **Data Processing:** pandas (pd) and numpy (np) are imported for data manipulation and numerical operations.

    Geospatial Mapping: folium, HeatMap, and MarkerCluster are used to create interactive maps for location analysis.

    Visualization: matplotlib, seaborn, and plotly.express are used to generate charts and visual insights.

[27]:

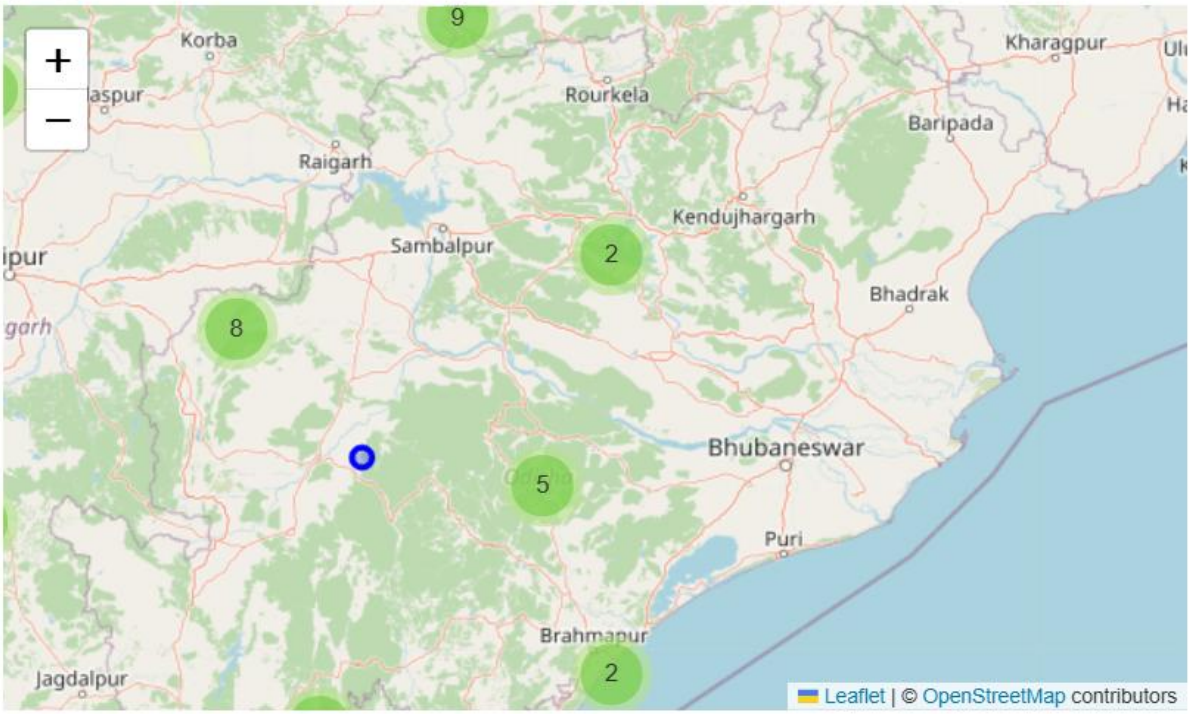
```
import pandas as pd
import numpy as np

import folium
from folium.plugins import HeatMap, MarkerCluster
import branca.colormap as cm

import plotly.express as px

import geopandas as gpd

import matplotlib.pyplot as plt
import seaborn as sns
```

```
m2 = folium.Map(
    location=[stations.latitude.mean(),
              stations.longitude.mean()],
    zoom_start=11
)

marker_cluster = MarkerCluster().add_to(m2)

for _, row in stations.iterrows():
    popup_text = f"""
    <b>Station ID:</b> {row.station_id}<br>
    <b>Total Ports:</b> {row.total_ports}<br>
    <b>Fast Ports:</b> {row.fast_ports}<br>
    <b>Capacity (kW):</b> {row.station_capacity_kw}
    """
```

```
    folium.CircleMarker(
        location=[row.latitude, row.longitude],
        radius=5,
        popup=popup_text,
        color="blue",
        fill=True
    ).add_to(marker_cluster)

m2
```

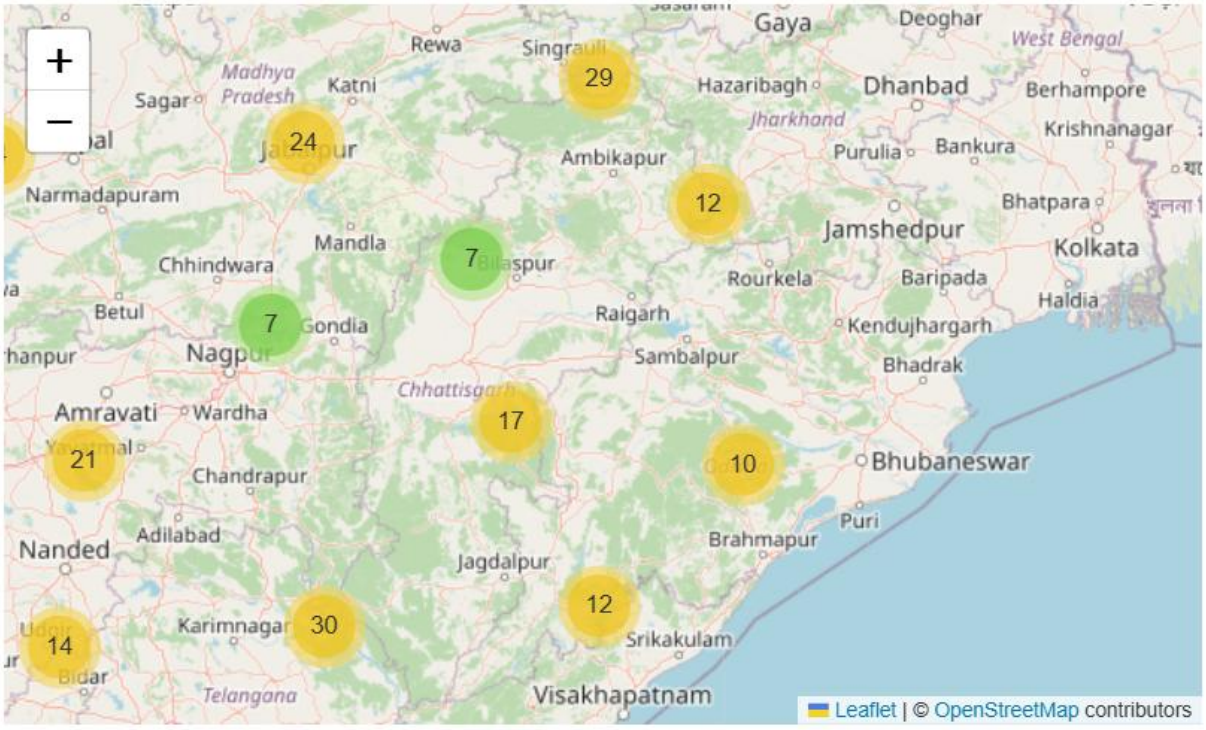
Results :

[30]:



Cumulative Output :

[30]:



12. **Data Selection:** Relevant columns are selected and missing values are removed using dropna() to ensure clean data for analysis.

3D Visualization: A 3D plot is created using Axes3D and ax.scatter(x, y, z) where **x = energy_consumed_kwh**, **y = grid_load_percentage**, and **z = charging_cost**.

Color Mapping: The cmap='viridis' and colorbar() functions are used to represent variations in charging cost through color intensity.

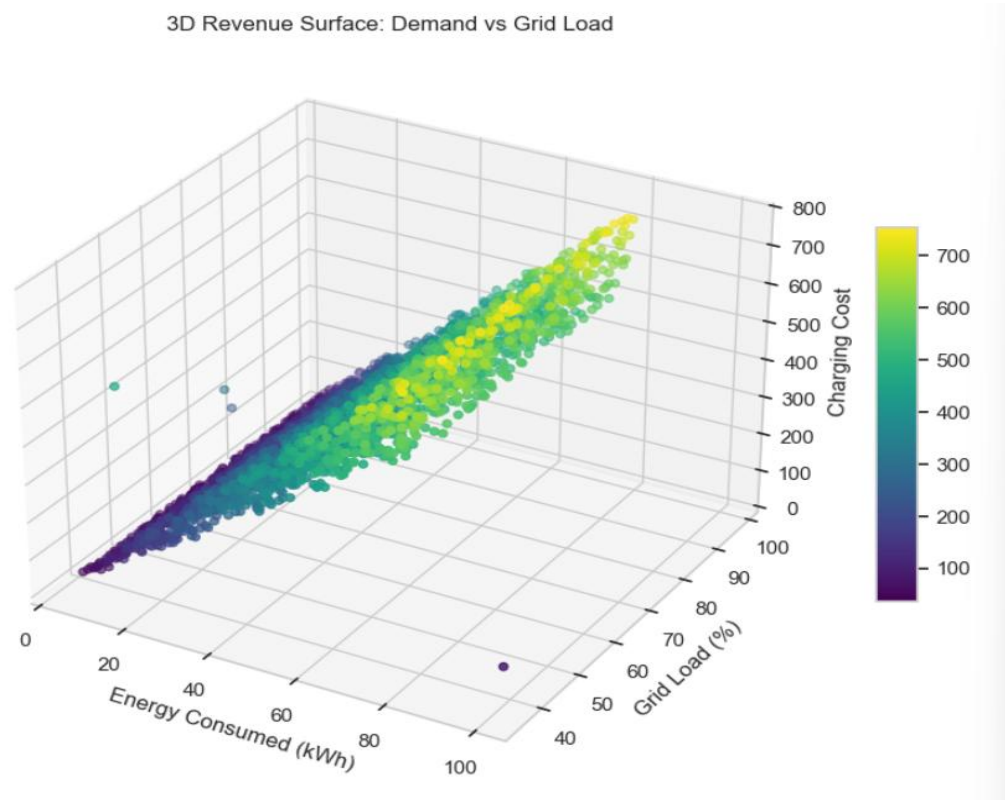
[48]:

```
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm
```

[49]:

```
# Select relevant columns
df = sessions[['energy_consumed_kwh',
               'grid_load_percentage',
               'charging_cost']].dropna()

# Create figure
fig = plt.figure(figsize=(12,8))
ax = fig.add_subplot(111, projection='3d')

x = df['energy_consumed_kwh']
y = df['grid_load_percentage']
z = df['charging_cost']
\
scatter = ax.scatter(x, y, z, c=z, cmap='viridis')

ax.set_xlabel("Energy Consumed (kWh)")
ax.set_ylabel("Grid Load (%)")
ax.set_zlabel("Charging Cost")
ax.set_title("3D Revenue Surface: Demand vs Grid Load")

fig.colorbar(scatter, shrink=0.5, aspect=10)

plt.show()
```

Integrated Result :



13 . Selecting important features (energy, duration, grid load, cost) and clean data using .dropna()

Normalized data using StandardScaler() so all features are on the same scale

Applying KMeans() to group charging sessions into 4 clusters (behavior patterns)

Using PCA() to reduce data into 2 dimensions for visualization (PCA1, PCA2)

Ploted clusters using plt.scatter() where colors represent different charging behavior groups

[55]:

```
# EV DEMAND REGIME DETECTION + CLUSTERED BEHAVIOR PHASE MAP

# This graph will:
# Detect hidden behavioral regimes in charging sessions
# Cluster sessions into demand patterns
# Reduce dimensions using PCA
# Visualize behavioral phase separation

This is unsupervised learning visualization.
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
```

[56]:

```
features = sessions[['energy_consumed_kwh',
                     'session_duration_minutes',
                     'grid_load_percentage',
                     'charging_cost']].dropna()

# Standardize
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)
```

[57]:

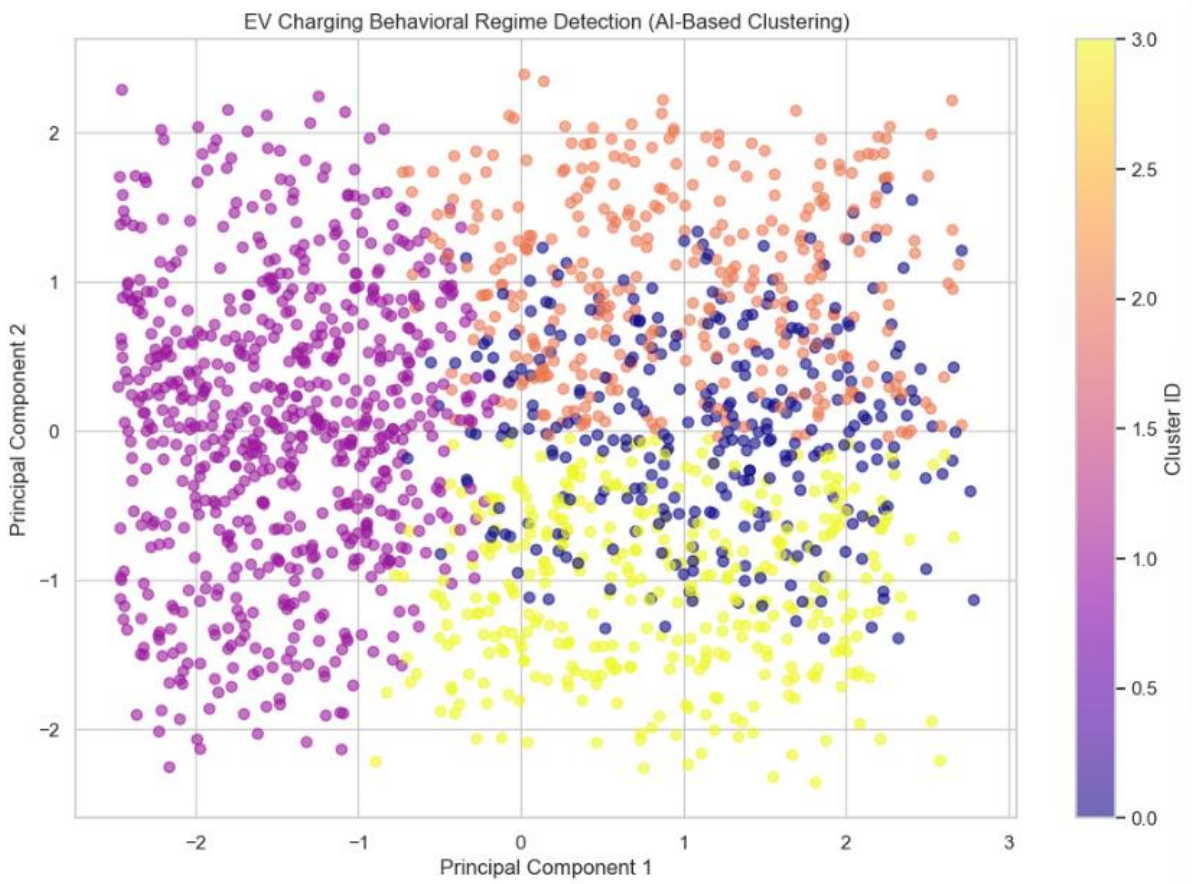
```
kmeans = KMeans(n_clusters=4, random_state=42)
clusters = kmeans.fit_predict(scaled_features)

features['cluster'] = clusters
```

[58]:

```
pca = PCA(n_components=2)
components = pca.fit_transform(scaled_features)

features['PCA1'] = components[:,0]
features['PCA2'] = components[:,1]
```

[59]:

```
plt.figure(figsize=(12,8))

scatter = plt.scatter(
    features['PCA1'],
    features['PCA2'],
    c=features['cluster'],
    cmap='plasma',
    alpha=0.6
)

plt.title("EV Charging Behavioral Regime Detection (AI-Based Clustering)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.colorbar(scatter, label="Cluster ID")
plt.show()
```



- ## SQL Powered Data Analytics

Library Installation & Import: The command pip install pandas ensures the Pandas library is available, and libraries like pandas and sqlite3 are imported for data handling and database operations.

Data Loading: CSV files are loaded into DataFrames using pd.read_csv() for tables such as cities, stations, vehicles, and charging sessions.

Database Storage: The DataFrames are stored in an SQLite database using to_sql() to create tables like cities, charging_stations, ev_vehicles, and charging_sessions.

1. Followed by loading datasets :

```
!pip install pandas
```

```
Requirement already satisfied: pandas in c:\users\sahoo\anaconda3\lib\site-packages (2.3.3)
Requirement already satisfied: numpy>=1.26.0 in c:\users\sahoo\anaconda3\lib\site-packages (from pandas) (2.3.5)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\sahoo\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\sahoo\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\sahoo\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\sahoo\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
import pandas as pd
import sqlite3
```

```
cities = pd.read_csv("Proj1_City.csv")
stations = pd.read_csv("Proj1_Charging_Stations.csv")
vehicles = pd.read_csv("Proj1_Vechiles.csv")
sessions = pd.read_csv("Proj1_Charging_Slots.csv")
```

```
conn = sqlite3.connect("ev_database.db")
```

```
cities.to_sql("cities", conn, if_exists="replace", index=False)
stations.to_sql("charging_stations", conn, if_exists="replace", index=False)
vehicles.to_sql("ev_vehicles", conn, if_exists="replace", index=False)
sessions.to_sql("charging_sessions", conn, if_exists="replace", index=False)
```

2. Fetching the adaptation rates :

```
pd.read_sql_query("SELECT city_name, ev_adoption_rate FROM cities", conn)
```

	city_name	ev_adoption_rate
0	Mumbai	0.18
1	Delhi	0.15
2	Bangalore	0.25
3	Bhubaneswar	0.34
4	Hyderabad	0.20
5	Pune	0.22
6	Ahmedabad	0.16
7	Kolkata	0.14

3. Detecting stations with fast charging dominance :

Stations with hight number of fast charging stations

```
# Detect Stations With Fast Charging Dominance
pd.read_sql_query("""
SELECT station_id,
total_ports,
fast_ports,
CASE
    WHEN fast_ports > total_ports/2 THEN 'Fast Charging Dominant'
    ELSE 'Normal Station'
END AS station_type
FROM charging_stations
""", conn)
```

	station_id	total_ports	fast_ports	station_type
0	1	17	14	Fast Charging Dominant
1	2	17	6	Normal Station
2	3	10	2	Normal Station
3	4	17	8	Normal Station
4	5	11	10	Fast Charging Dominant
...	...	...	...	...
1003	24	11	2	Normal Station
1004	25	12	5	Normal Station
1005	16	16	8	Normal Station
1006	17	15	2	Normal Station
1007	18	9	9	Fast Charging Dominant

1008 rows × 4 columns

4. Grid stress detection :

Distributing session ids on the basis of max grid load percentage.

```
# Grid Stress Detection
pd.read_sql_query("""
SELECT session_id,
station_id,
grid_load_percentage,
CASE
    WHEN grid_load_percentage >= 80 THEN 'Critical Load'
    WHEN grid_load_percentage BETWEEN 60 AND 79 THEN 'Moderate Load'
    ELSE 'Safe Load'
END AS grid_status
FROM charging_sessions
ORDER BY grid_load_percentage DESC
""", conn)
```

	session_id	station_id	grid_load_percentage	grid_status
0	1108	993	97.99	Critical Load
1	1777	283	97.98	Critical Load
2	743	777	97.96	Critical Load
3	1298	114	97.96	Critical Load
4	446	743	97.95	Critical Load
...	...	...	...	...
3001	338	277	35.13	Safe Load
3002	1090	988	35.12	Safe Load
3003	1059	284	35.09	Safe Load
3004	2062	615	35.07	Safe Load
3005	739	404	35.05	Safe Load

3006 rows × 4 columns

5. Vehicle Energy Efficiency impact :

```
# Vehicle Energy Efficiency Impact
pd.read_sql_query("""
SELECT v.manufacturer,
SUM(s.energy_consumed_kwh) AS total_energy_consumed,
AVG(v.efficiency_km_per_kwh) AS avg_efficiency
FROM charging_sessions s
JOIN ev_vehicles v
ON s.vehicle_id = v.vehicle_id
GROUP BY v.manufacturer
ORDER BY total_energy_consumed DESC
""", conn)
```

	manufacturer	total_energy_consumed	avg_efficiency
0	Ola Electric	40966.99	26.257108
1	Mahindra	38222.75	31.894762
2	Tata Motors	32651.15	28.084404
3	Ather Energy	25202.93	25.803378

6. Top vehicle manufacturer per city : (Ranking on maximum charging session per month)

```
# Top Vehicle Manufacturer per City
# (Partition Ranking)
# Which EV manufacturer dominates charging sessions in each city?
pd.read_sql_query("""
SELECT *
FROM (
    SELECT
    c.city_name,
    v.manufacturer,
    COUNT(s.session_id) AS total_sessions,
    RANK() OVER(
        PARTITION BY c.city_name
        ORDER BY COUNT(s.session_id) DESC
    ) AS rank_in_city
    FROM charging_sessions s
    JOIN ev_vehicles v
    ON s.vehicle_id = v.vehicle_id
    JOIN cities c
    ON s.city_id = c.city_id
    GROUP BY c.city_name, v.manufacturer
)
WHERE rank_in_city = 1
""", conn)
```

	city_name	manufacturer	total_sessions	rank_in_city
0	Ahmedabad	Ola Electric	102	1
1	Bangalore	Ola Electric	114	1
2	Bhubaneswar	Mahindra	109	1
3	Delhi	Ola Electric	107	1
4	Hyderabad	Mahindra	99	1
5	Kolkata	Ola Electric	118	1
6	Mumbai	Tata Motors	107	1
7	Pune	Mahindra	110	1

7. Grid load renewable charging : (Distribution into high , medium and low loads)

```
# Grid Load vs Renewable Charging
pd.read_sql_query("""
SELECT
CASE
WHEN grid_load_percentage > 80 THEN 'High Load'
WHEN grid_load_percentage BETWEEN 50 AND 80 THEN 'Medium Load'
ELSE 'Low Load'
END AS grid_category,
AVG(renewable_ratio) AS avg_renewable_usage,
COUNT(session_id) AS sessions
FROM charging_sessions
GROUP BY grid_category
""", conn)
```

	grid_category	avg_renewable_usage	sessions
0	High Load	37.747993	876
1	Low Load	38.034693	688
2	Medium Load	37.263278	1442

8. Percentage of energy consumption : (Max energy sum consumed to a particular segment)

```
# EV Segment Contribution to Charging
pd.read_sql_query("""
SELECT
v.segment,
COUNT(s.session_id) AS sessions,
SUM(s.energy_consumed_kwh) AS energy
FROM charging_sessions s
JOIN ev_vehicles v
ON s.vehicle_id = v.vehicle_id
GROUP BY v.segment
ORDER BY energy DESC
""", conn)
```

	segment	sessions	energy
0	hatchback	413	26264.60
1	suv	428	18270.37
2	ebike	408	17096.35
3	muv	397	16828.09
4	sedan	390	16540.45
5	scooter	376	15944.33
6	passenger	287	13137.51
7	heavyduty	307	12962.12

9. Rolling energy consumption trend :

The query is executed on the charging_sessions table to analyze energy usage over time.

Window Function: The cumulative energy consumption is calculated using the window function **SUM(energy_consumed_kwh) OVER (ORDER BY start_time)**.

```
# Rolling Energy Consumption Trend
# (Window Running Total)
pd.read_sql_query("""
SELECT
start_time,
energy_consumed_kwh,
SUM(energy_consumed_kwh)
OVER (ORDER BY start_time)
AS cumulative_energy
FROM charging_sessions
""", conn)
```

	start_time	energy_consumed_kwh	cumulative_energy
0	01-01-2025 06:10	25.73	25.73
1	01-01-2025 06:20	6.91	32.64
2	01-01-2025 06:30	45.73	78.37
3	01-01-2025 06:40	28.47	106.84
4	01-01-2025 06:50	46.04	152.88
...	...	...	...
3001	22-01-2025 01:20	23.58	136953.99
3002	22-01-2025 01:30	NaN	136953.99
3003	22-01-2025 01:40	16.38	136970.37
3004	22-01-2025 01:50	23.10	136993.47
3005	22-01-2025 02:00	50.35	137043.82

3006 rows × 3 columns

10. Top charging station that generate the highest revenue : The query uses WITH station_revenue AS (...) to create a temporary table that summarizes charging station performance.

Aggregation Functions: Metrics are calculated using formulas like COUNT(session_id) for total sessions, SUM(charging_cost) for total revenue, and AVG(grid_load_percentage) for average grid load.

City-Level Analysis: Another CTE calculates the city average revenue using AVG(total_revenue) grouped by city_name to compare station performance within each city.

```
#Find the Top 5 Charging Stations that generate the highest revenue and classify them as
# High / Medium / Low performing stations within their city.
query = """

WITH station_revenue AS (
    SELECT
        cs.station_id,
        c.city_name,
        COUNT(cs.session_id) AS total_sessions,
        SUM(cs.energy_consumed_kwh) AS total_energy,
        SUM(cs.charging_cost) AS total_revenue,
        AVG(cs.grid_load_percentage) AS avg_grid_load
    FROM charging_sessions cs
    JOIN charging_stations s
        ON cs.station_id = s.station_id
    JOIN cities c
        ON cs.city_id = c.city_id
    GROUP BY cs.station_id, c.city_name
),

city_avg AS (
    SELECT
        city_name,
        AVG(total_revenue) AS city_avg_revenue
    FROM station_revenue
    GROUP BY city_name
),
```

Ranking Calculation: The query uses the window function **RANK() OVER (ORDER BY total_revenue DESC)** to rank charging stations based on their total revenue.

Performance Classification: A **CASE WHEN** condition categorizes stations into groups such as *High*, *Above Average*, or *Needs Improvement* depending on their revenue compared to the city average.

Top Station Selection: The final query filters results using WHERE revenue_rank <= 5 to display the **top 5 performing charging stations**.

```
ranked_stations AS (
    SELECT
        sr.station_id,
        sr.city_name,
        sr.total_sessions,
        sr.total_energy,
        sr.total_revenue,
        sr.avg_grid_load,
        ca.city_avg_revenue,

        RANK() OVER (ORDER BY sr.total_revenue DESC) AS revenue_rank,

        CASE
            WHEN sr.total_revenue > ca.city_avg_revenue * 1.5 THEN 'High Performer'
            WHEN sr.total_revenue > ca.city_avg_revenue THEN 'Above Average'
            ELSE 'Needs Improvement'
        END AS performance_category

    FROM station_revenue sr
    JOIN city_avg ca
        ON sr.city_name = ca.city_name
)

SELECT
    station_id,
    city_name,
    total_sessions,
    total_energy,
    total_revenue,
    avg_grid_load,
    revenue_rank,
    performance_category
FROM ranked_stations
WHERE revenue_rank <= 5;

"""

pd.read_sql_query(query, conn)
```

	station_id	city_name	total_sessions	total_energy	total_revenue	avg_grid_load	revenue_rank	performance_category
0	764	Ahmedabad	5	265.98	2228.16	80.770000	1	High Performer
1	23	Ahmedabad	4	271.86	2158.20	74.500000	2	High Performer
2	281	Hyderabad	4	233.99	1841.67	54.567500	3	High Performer
3	997	Mumbai	3	202.90	1706.98	81.643333	4	High Performer
4	136	Kolkata	3	194.02	1665.25	64.516667	5	High Performer

- ## Power BI Visuals

We have now transitioned our analysis from SQL exploration to an interactive Power BI dashboard to gain deeper, visual insights into EV charging patterns, efficiency, and market behaviour.

Brand Stats

Provides a comparative view of EV manufacturers like Mahindra, Tata, Ather, and Ola based on usage, battery capacity, and performance metrics.

Utilities

Focuses on operational efficiency across cities and vehicle segments, including : Energy consumed , Charging cost , Grid load impact , Renewable energy usage

Utilities (Efficiency)

Highlights efficiency trends by comparing battery capacity, energy consumption, and renewable ratios.

Utilities (Market)

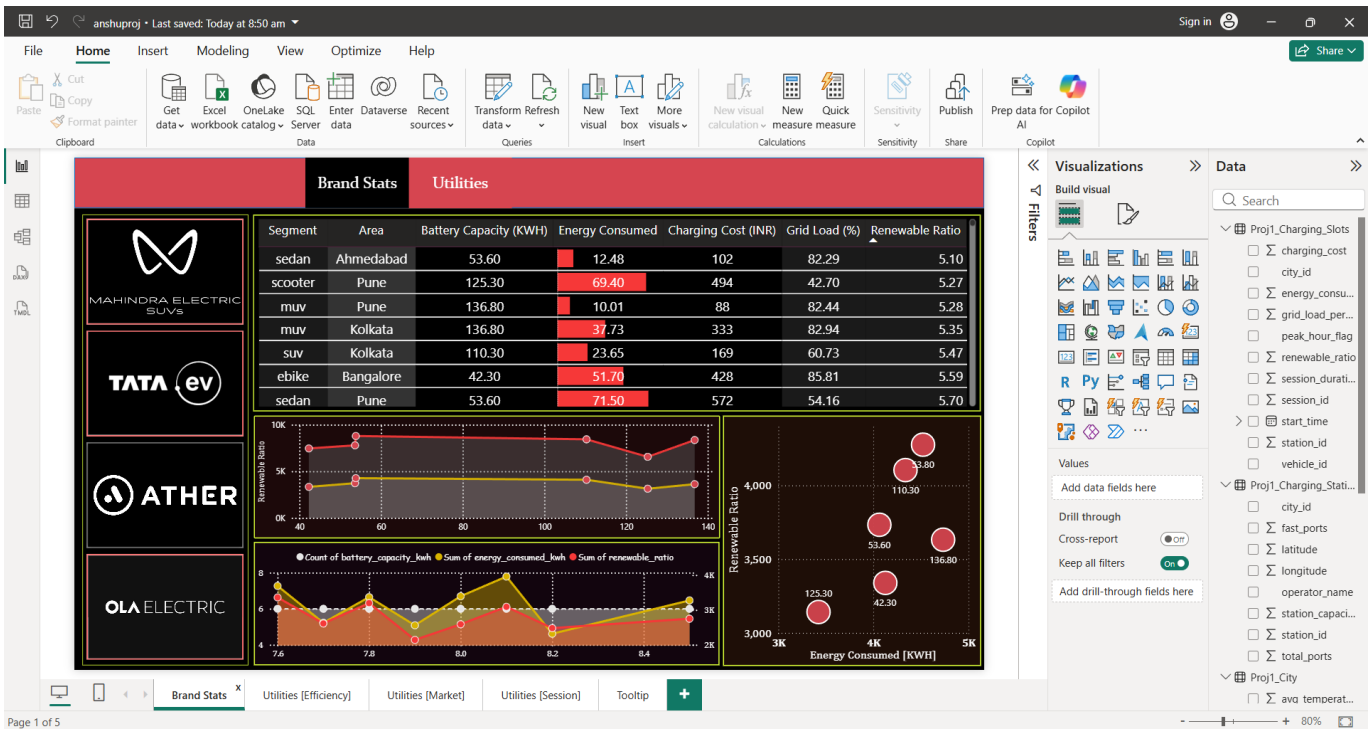
Analyses demand distribution, identifying which segments and cities contribute most to EV charging activity.

Utilities (Session)

Provides detailed session-level insights such as duration, peak hours, and usage behaviour.

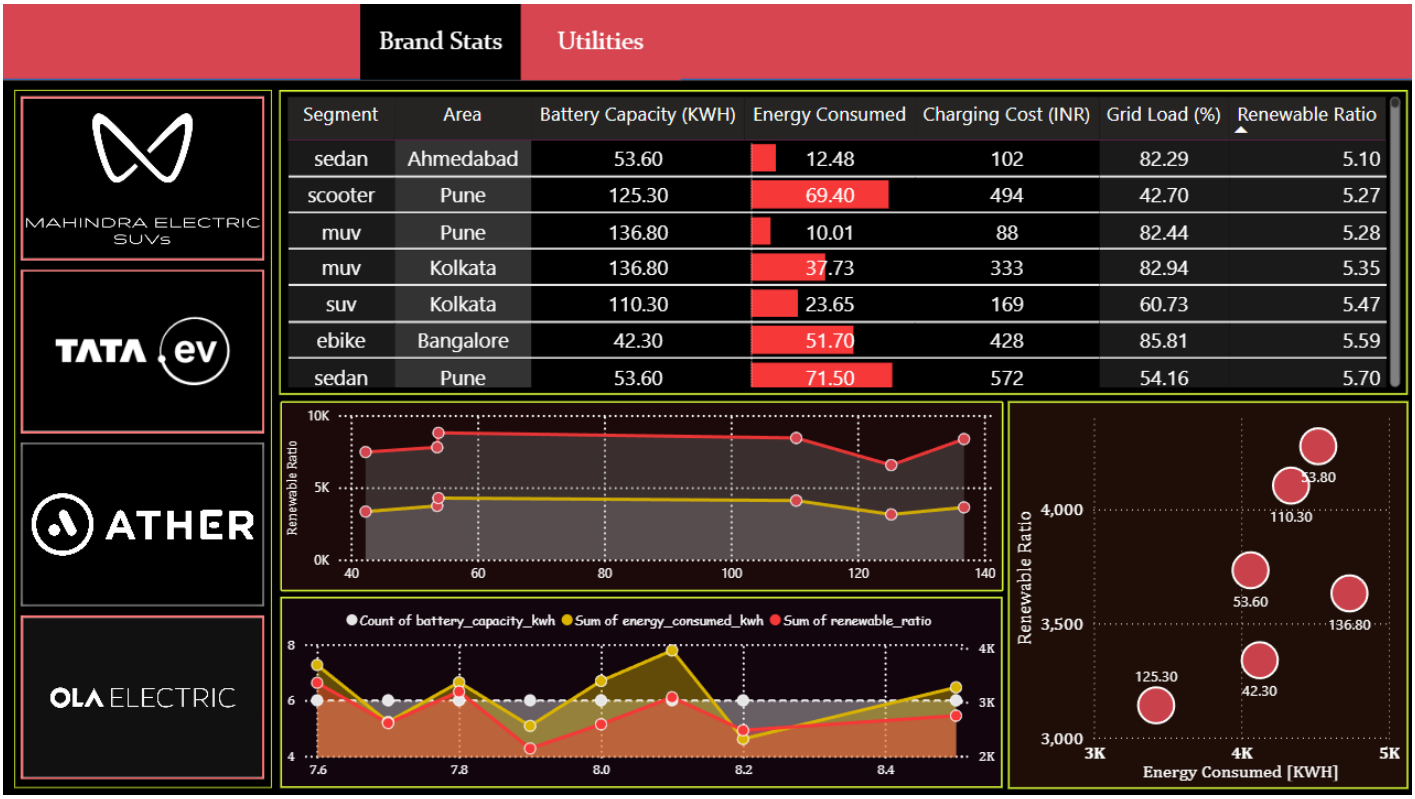
Tooltip (Interactive Insight Layer)

Custom tooltip pages enhance user interaction by displaying detailed breakdowns when hovering over visuals, enabling quick micro-level insights without navigating away.



1. Brand Stats [Manufacturer Insights Dashboard (Proj1_Vehicles)] :

This page provides a comprehensive overview of vehicle performance and usage metrics categorized by manufacturer. It enables users to analyze key operational and efficiency indicators across different segments and locations.



Key Functionalities:

- **Button Slicer (Manufacturer Selection) :**
  An interactive button slicer allows users to filter the entire dashboard based on selected manufacturers for focused analysis.

- **Descriptive Data Table:**
  Displays detailed attributes including Vehicle Segment , Area (City) , Battery Capacity (kWh) , Energy Consumed (kWh) , Charging Cost (INR) , Grid Load (%) ,Renewable Ratio

- **Stacked Area Chart :**
  Visualizes the relationship between Energy Consumption , Renewable Ratio . It highlights trends and variations across different data points, making it easier to identify efficiency patterns.

- **Comparative Visuals (Line & Scatter) :**
  Additional charts provide insights into trends across battery capacity and energy usage , Correlation between energy consumption and renewable ratio.

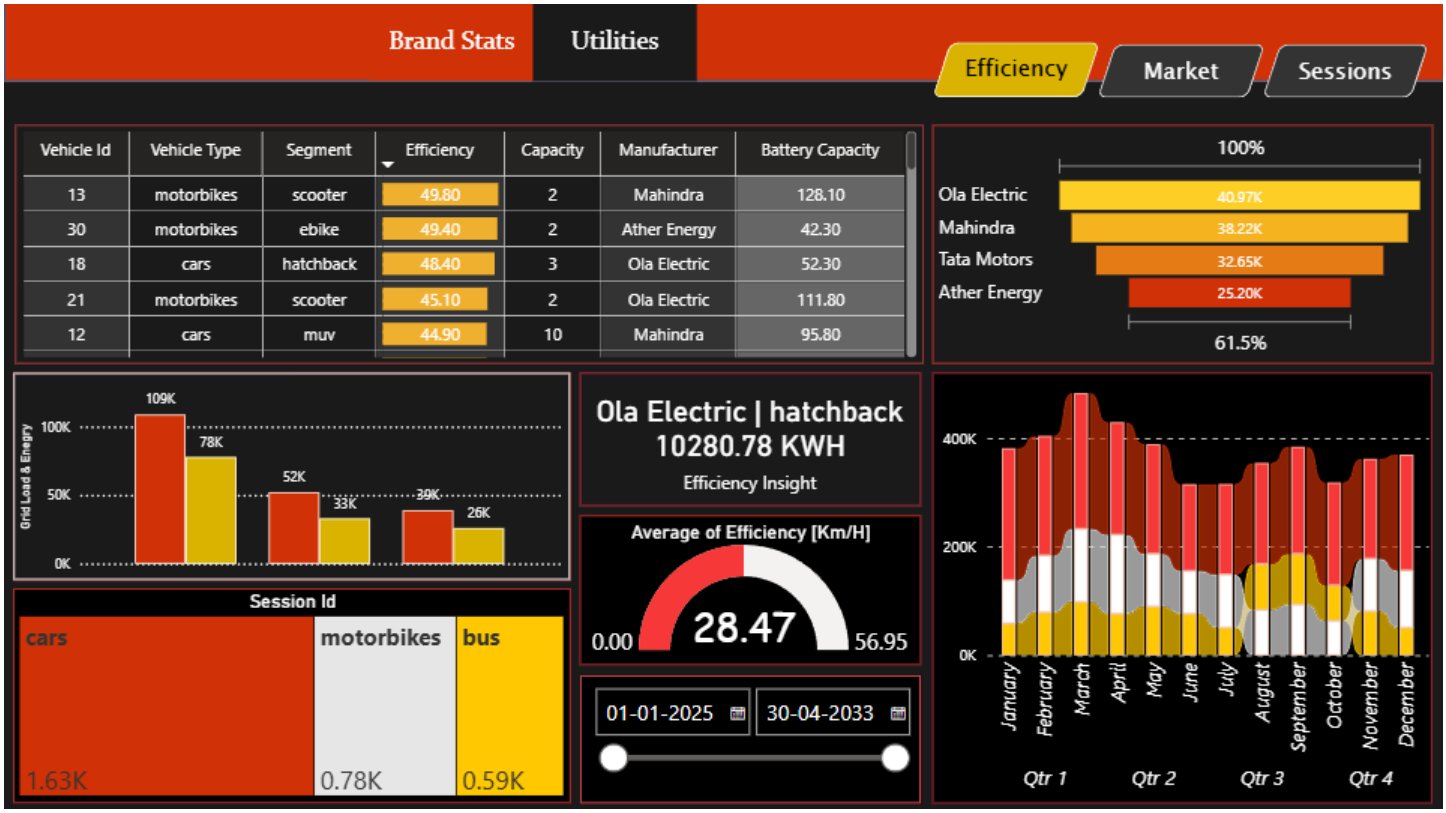
---

2. Utilities [Efficiency Analysis Dashboard] :

This page focuses on analyzing vehicle efficiency, performance trends, and session distribution across different manufacturers and vehicle types. It helps in identifying high-performing vehicles and understanding usage patterns over time.

Key Functionalities :

- **Toggle Buttons (Efficiency / Market / Sessions):**
  Interactive navigation buttons allow users to switch between different analytical views, with the current focus on Efficiency insights.
- **Date Range Slicer:**
  Allows users to filter the dashboard based on a specific time period for dynamic and time-based analysis.
- **Vehicle Performance Table:**
  Displays detailed vehicle-level data including Vehicle ID, Vehicle Type, Segment, Efficiency, Capacity, Manufacturer, and Battery Capacity.
- **Manufacturer Comparison Bar Chart :**
  Compares total performance metrics across manufacturers like Ola Electric, Mahindra, Tata Motors, and Ather Energy, enabling quick benchmarking.
- **Gauge Chart (Average Efficiency) [DAX] :**
  Displays the average efficiency (Km/H) with a visual indicator, helping users understand overall performance against defined thresholds.



- **Grid Load Energy Bar Chart:**
  Shows energy distribution across categories, enabling comparison of grid load usage among different segments.
- **Session Distribution (Treemap) :**
  Visualizes session count distribution across vehicle types (cars, motorbikes, buses), helping identify dominant usage categories.
- **Monthly Trend Analysis (Stacked Column Chart) :**
  Displays monthly variations in performance metrics across quarters, helping in identifying seasonal or periodic trends.

---

2. Utilities [Market Insights Dashboard (Proj1_Vehicles)] :

This page provides a detailed view of vehicle distribution, session activity, and energy usage patterns across different vehicle types and segments. It enables users to evaluate market trends, manufacturer performance, and energy efficiency insights .

Key Functionalities :

• Navigation Tabs (Efficiency | Market | Sessions):
Interactive tab buttons allow users to switch between different analytical views, with the *Market* tab currently active for focused insights on vehicle distribution and usage.

• Date Range Slicer:
An interactive slider allows users to filter data between specific time periods, enabling temporal analysis of trends and performance.

• Descriptive Data Table:
Displays detailed vehicle-level information including *Vehicle ID, Vehicle Type, Segment, Total Session IDs, Renewable Ratio,* and *Manufacturer*.

• Session Distribution Bar Visualization:
A horizontal bar chart represents total session counts across vehicle types such as *cars, motorbikes,* and *buses*, helping identify dominant categories in usage.

- **Grid Load vs Energy Comparison Chart:**

A clustered column chart compares *Grid Load Energy* and *Energy Consumption* across segments, enabling analysis of infrastructure demand versus actual usage.

- **Efficiency Insight Card :**

Highlights key performance metrics for a selected manufacturer and segment (e.g., *Mahindra | SUV*), displaying total energy consumption (in kWh) along with a quick efficiency summary.

- **Gauge Chart (Energy Consumed) :**

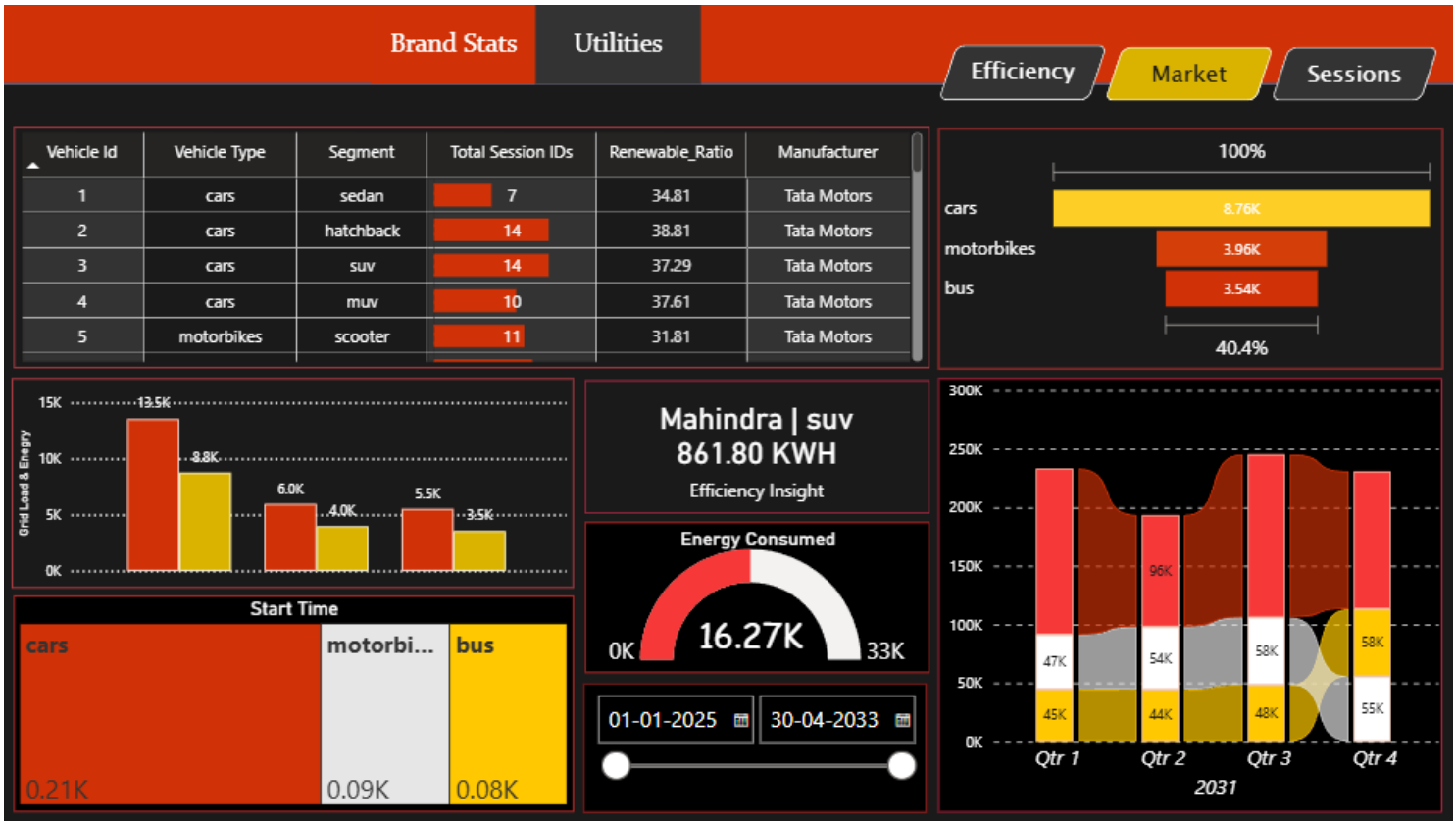
A semi-circular gauge visualizes total energy consumed against a defined scale, providing an intuitive understanding of consumption levels.

- **Start Time Distribution (Treemap) :**

A tree map shows the distribution of sessions by vehicle type (*cars, motorbikes, buses*), giving a proportional view of activity volume.

- **Quarterly Energy Trend (Stacked Area Chart) :**

Displays energy consumption trends across quarters (Q1–Q4), segmented by categories. This helps in identifying seasonal variations and consumption patterns over time.



---

3. Utilities [Session Insights Dashboard (Proj1_Vehicles)] :

This page focuses on analyzing session duration patterns across vehicle types, segments, and manufacturers, helping users understand usage behavior and performance trends.

Key Functionalities:

• Button Slicer (View Selection):

An interactive slicer at the top allows users to switch between *Efficiency, Market,* and *Session* views, enabling focused analysis.

• Descriptive Data Table:

Displays details like *Vehicle Type, Segment, Avg Session Duration, Capacity, Manufacturer,* and *Battery*.

• Session Distribution Chart:

Shows session duration trends across different segments.

• Session Ranking Bar Chart:

Highlights average session duration across vehicle types.

• Insight Card & Gauge:

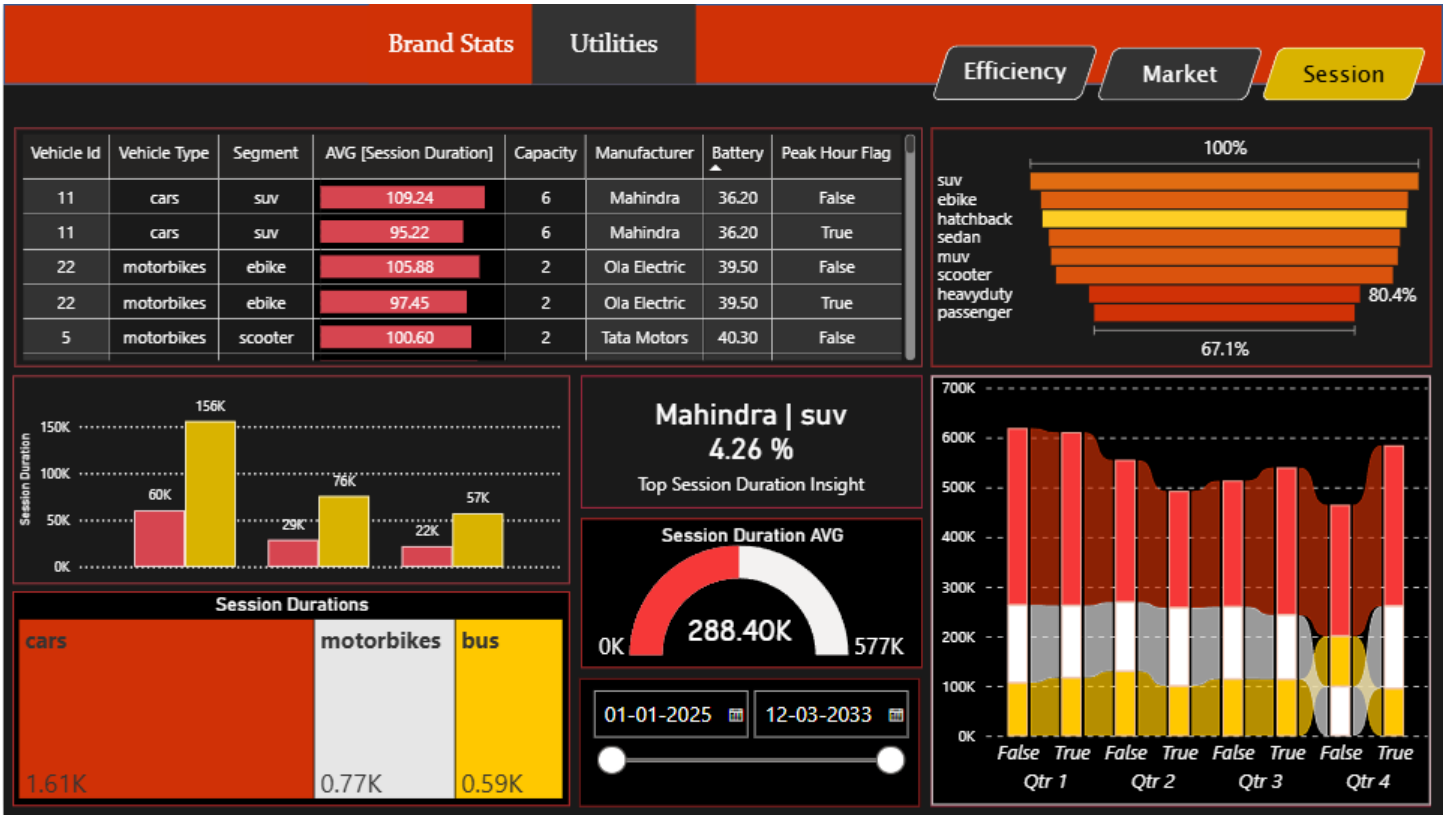
Displays key metrics for selected segment (e.g., *Mahindra | SUV*) with average session duration.

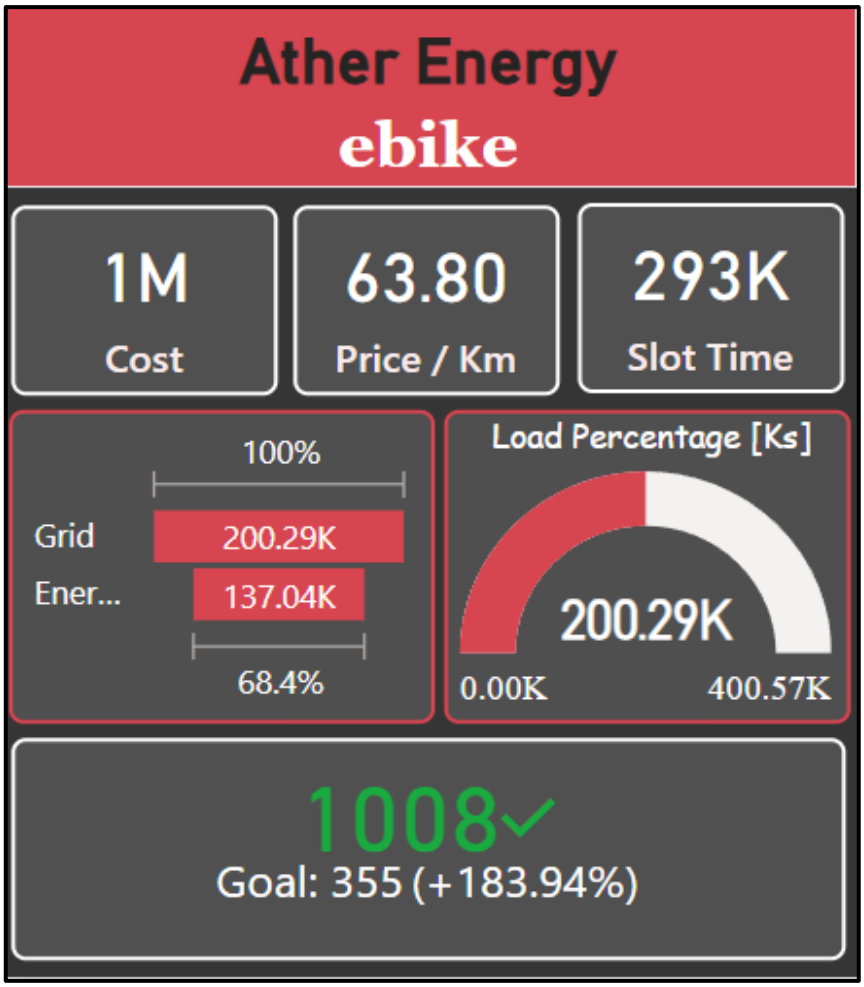
• Treemap (Session Durations):

Represents proportion of total session durations by vehicle type.

• Quarterly Trend Chart:

Visualizes session patterns across quarters with peak hour insights.





Tooltip [Manufacturer Quick Insight] :

This tooltip provides a quick summary of key performance metrics for a selected manufacturer and vehicle segment when users hover over visuals.

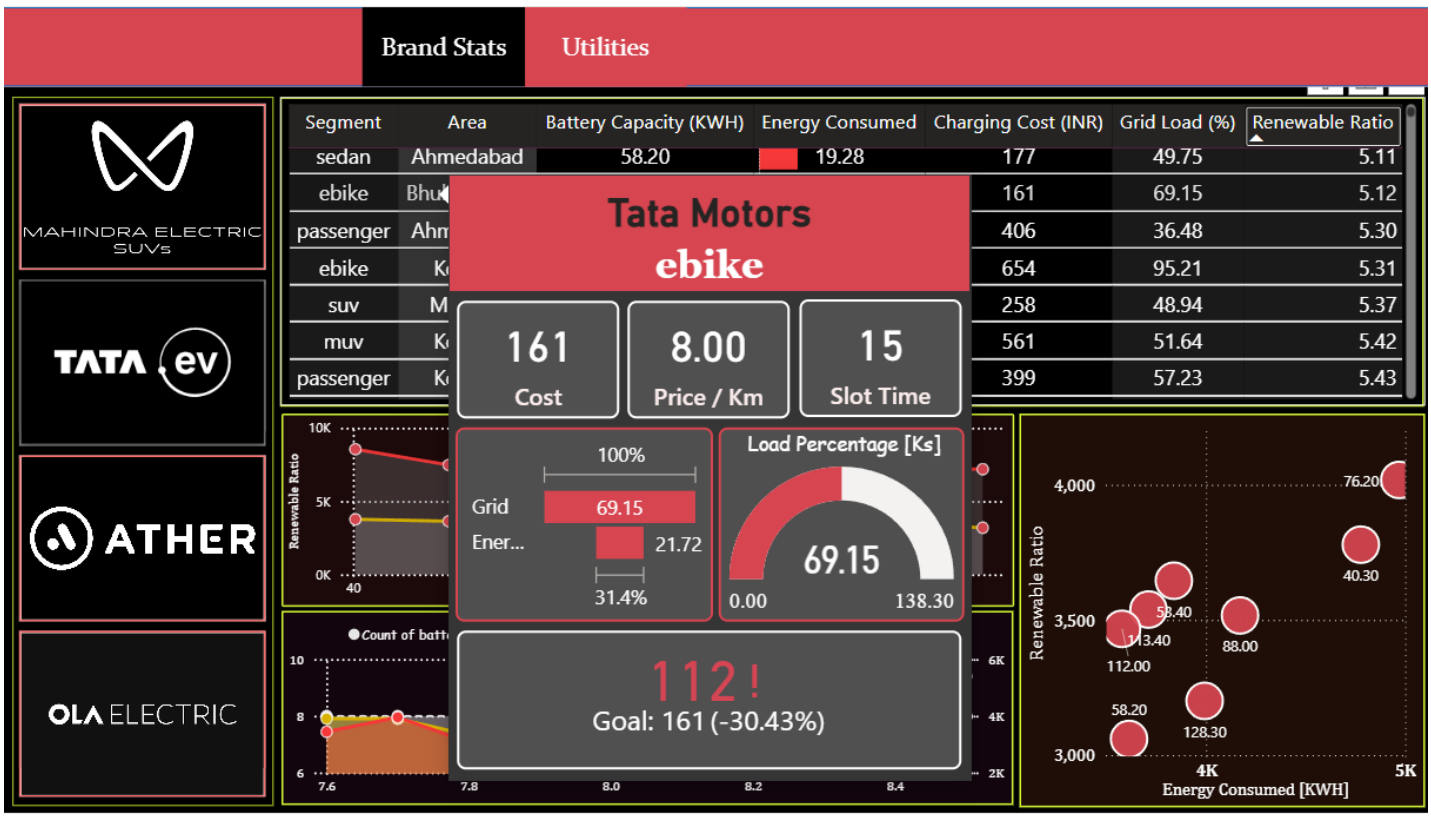
Key Functionalities:

- **Dynamic Context Display:** Updates based on the selected *manufacturer* and *segment* (e.g., *Tata Motors – ebike*).
- **KPI Summary Cards:** Shows *Cost, Price per Km,* and *Slot Time* for quick evaluation.

- **Grid vs Energy Comparison :**
Displays *Grid Load* and *Energy Consumption* with percentage comparison.

- **Gauge Chart (Load %) :**
Visualizes load utilization against total capacity.

- **Goal Indicator :**
Highlights actual performance against target with variance.



Summary :

1. Python – Data Processing & Analysis

Python is used for data cleaning, preprocessing, and exploratory analysis.

- Handled **missing values, duplicates, and text standardization**
- Performed **outlier detection (IQR method)**
- Generated **correlation heatmaps and visualizations**
- Applied **clustering (K-Means) and PCA** to identify charging behaviour patterns.
- Created advanced visuals like **3D plots and geospatial maps**

---

2. SQL – Data Management & Analytics

SQL is used to store, organize, and analyze structured data.

- Created relational tables for **sessions, vehicles, stations, and cities**
- Performed queries for:
  - **EV adoption trends**
  - **Grid load and stress analysis**
  - **Top manufacturers and stations**
  - **Energy consumption patterns**
- Used advanced concepts like **CTEs, window functions, and ranking**
- Identified **top-performing stations and revenue insights**.

---

3. Power BI – Visualization & Dashboarding

Power BI is used to build interactive dashboards for business insights.

- Developed dashboards like:
  - **Brand Stats (Manufacturer Analysis)**
  - **Utilities (Efficiency, Market, Session Insights)**
- Implemented features such as:
  - **Slicers and filters for dynamic analysis**
  - **KPI cards, gauge charts, and treemaps**
  - **Trend and comparison visuals**
- Added **custom tooltips** for detailed hover-based insights.